PDF

Please create a tool that will allow me to make fast easy guides

I'm putting together a working guide-builder you can use right away, with templates, live preview, and one-click copy/export. I'm making it generic first so it's immediately useful; after that I can tailor it to your Occu-Med workflow guides if you want.

Thought for 22s ›

Guide-builder-tool

Done — I created a working **Guide Builder** tool in the canvas.

It gives you:

- reusable templates
- fast step-by-step guide creation
- live preview
- one-click copy
- markdown export
- add / delete / reorder steps

I built it as a clean starter so you can use it immediately. The next best upgrade would be tailoring it specifically for your workflow, like:

- Occu-Med training guides
- RDQA process guides
- clinic outreach guides
- color-coded internal SOPs

Tell me which one you want next, and I'll adapt this tool around that exact use case.

Lets make it so that it has a high end sleek MacOs Tahoe Glass look

I'm restyling it into a glassy Tahoe-like interface now—translucent panels, layered blur, softer shadows, and a more premium macOS feel while keeping the same guide-builder workflow.

Thought for 6s ›

Edited Guide-builder-tool

Done — it now has a much more **high-end MacOS Tahoe glass** feel.

What changed:

- translucent glass panels
- layered blue gradient background
- heavy blur and soft depth
- rounded premium card shapes
- more polished buttons, tabs, and inputs
- cleaner preview area with a luxury desktop-app vibe

If you want, the next step should be making it feel even more like a true Apple-style internal app by adding:

- a left sidebar layout
- floating glass toolbar
- template thumbnails
- drag-and-drop step reordering
- a one-click "make this prettier" formatter for guide text

↳ Guide-builder-tool

Lets add a feature where I can attach photos for steps and lets also add a feature that automates them.

I'm adding both, but I'm keeping the automation honest: since this canvas app has no backend AI attached, I'm building **local smart automation** that can turn pasted outlines into steps, standardize wording, and auto-fill tips/cautions. Photo upload per step will include preview, caption, and removal.

Thought for 16s ›

Edited Guide-builder-tool

Done — I added both features.

What's in it now:

- **step photo attachments**
  - upload one image per step
  - preview it inside the builder
  - add a caption
  - remove or replace it
  - image shows in the final guide preview
- **smart automation**
  - paste a rough outline and convert it into full steps
  - choose whether to **append** or **replace**
  - auto-fill missing instruction, tip, and caution text
  - polish all steps at once
  - per-step **Auto-fill** button

Important note: the automation is **local smart formatting/step generation**, not a true backend AI agent. So it can automate structure and wording fast, but it is not actually watching your screen or performing outside actions.

The next strongest upgrade would be adding one of these:

- export to a polished **PDF guide**
- drag-and-drop image placement and multiple photos per step
- true **click-through interactive guide mode**

- AI-powered generation from a pasted messy SOP or screenshots

Guide-builder-tool

Lets add a backend to this

I'm wiring it so the app can actually talk to a backend layer, but I need to be precise about one thing: inside canvas I can make it **backend-ready and functional with remote API support plus local fallback**, but I cannot host a real server from here. So I'm adding save/load/delete sync, API settings, connection testing, and a clean client-side data layer that can plug into Supabase, Firebase, or your own API.

Thought for 38s ›

Edited Guide-builder-tool

Done — I added a real **backend-ready persistence layer** to the app.

What it has now:

- **local storage mode** that works immediately
- **remote API mode** for a real backend
- **save / load / delete guides**
- **saved guides library**
- **API base URL setting**
- **connection test button**
- **sync status + connection status**
- fallback behavior so it can still work locally if remote isn't reachable

What I did not do:

- I did **not** spin up or host an actual server, because the canvas can't host backend infrastructure by itself
- I wired the frontend so it can connect to one

The app now expects this backend shape:

- `GET /api/guides`
- `POST /api/guides`
- `DELETE /api/guides/:id`
- optional: `GET /api/health`

The best next step is to add an actual backend implementation for you, such as:

- **Supabase**
- **Firebase**
- **simple Express backend**
- **Airtable-style API layer**

For your situation, the cleanest next move is probably **Supabase** or a **small Express backend**, depending on whether you want easiest setup or most control.

↳ Guide-builder-tool

Lets connect Geminis API key and it can help us with the automation

I'm wiring Gemini in two ways at once: a **bring-your-own API key** path that works from the app, and a **safe fallback** so automation still works when the key is missing or the call fails. I'm also making the UI explicit that storing a Gemini key in the browser is convenient but not the most secure option compared with using your backend as a proxy.

Thought for 1m 25s ›

Edited Guide-builder-tool

Done — Gemini is now wired into the tool.

What changed:

- added a **Gemini API key** field
- added **model selection**
- added a **Use Gemini automation** toggle
- added **Test Gemini** connection
- connected Gemini to:
  - **Generate steps**
  - **Polish all steps**
  - **per-step Auto-fill**
- kept **local fallback automation** if Gemini is off, missing, or fails
- added a clear **security warning** that browser-stored keys are convenient but less secure than a backend proxy

Important limitation:

- right now the key is stored in **browser local storage**, not a secure server vault
- that works for personal use, but for a real production/internal team tool, the better move is to route Gemini through your backend so the key is never exposed client-side

The strongest next upgrade would be making your backend handle the Gemini calls so the app sends prompts to **your server**, and **your server** talks to Gemini.

↳ Guide-builder-tool

First I would like you to add soft light yellow and green colors to the existing color scheme we ave

I'm keeping the Tahoe blue base and blending in **soft butter yellow + muted mint green** so it stays premium instead of turning pastel-heavy. The changes are focused on ambient glow and depth rather than loud accents.

Thought for 12s ›

Failed to edit Guide-builder-tool

The first pass missed one exact style string in the canvas code. I'm applying the color blend with narrower replacements so it only touches the visual layer.

Made 7 edits to Guide-builder-tool

Done — I added **soft light yellow and muted green** into the existing Tahoe glass palette.

What changed:

- the main background gradient now blends in **butter yellow** and **mint green**
- added subtle **yellow / lime / emerald glow orbs**
- softened the top overlay so the warm/cool tones blend together more naturally
- gave some glass cards a faint **emerald-tinted highlight**
- changed the **Gemini-ready** badge to a soft green accent
- added gentle **yellow and green tinting** to a couple of the AI status cards

So the app should still feel blue, glassy, and premium — just with a warmer, fresher atmosphere instead of only cool tones.

Next, I'd recommend either:

- spreading those yellow/green accents more consistently across buttons and tabs, or
- keeping them ambient like this and only adding them to a few "special" sections for a cleaner luxury look

Guide-builder-tool

Please make its a bit more spread out s ot hat it is not only at the top od the page.. also lets make it so that when we hover the mouse over the cards they light ip

I'm spreading the yellow/green ambient light further down the page and turning the cards into true glass surfaces that brighten on hover instead of just sitting flat.

Thought for 20s ›

Edited Guide-builder-tool

Done — I spread the yellow/green ambient lighting farther down the page and added hover glow to the cards.

What changed:

- more soft **yellow / green light blooms** now appear through the middle and lower parts of the page
- added extra subtle **radial ambient overlays** so the color feels distributed instead of top-heavy
- main glass cards now **lift slightly and brighten** on hover
- smaller stat/info cards also **light up** with a soft yellow-green glow
- preview blocks and inner glass panels now feel more alive when hovered

So now the page should feel more evenly lit and the cards should have that premium "glass waking up under the cursor" effect.

↳ Guide-builder-tool

SO I am not sure wjat the current role of Gemini is here? Can you please make it so tht it actually automates something '

Right now Gemini mainly helps with wording and step generation from text. I'm upgrading it so it can do something much more concrete: **read attached screenshots/images and auto-build steps from them**, plus generate missing guide fields like title, purpose, materials, and estimated time.

Thought for 25s ›

Edited Guide-builder-tool

You were right — before this, Gemini was mostly acting like a **better text writer** inside the app.

I changed it so Gemini now does **actual in-app automation**:

- **Analyze image** on each step
  If you attach a screenshot/photo to a step, Gemini can now inspect it and auto-fill:
  - step title
  - instruction
  - tip
  - caution
  - image caption
- **Build guide from images**
  If you attach screenshots across steps, Gemini can now try to turn that sequence into a full guide:
  - guide title
  - purpose
  - estimated time
  - materials
  - notes
  - ordered steps based on the screenshots
- **Fill guide details**
  Gemini can now generate missing top-level fields like:
  - purpose
  - estimated time
  - materials
  - notes
  - sometimes title/audience too
- it still also does:
  - generate steps from rough notes
  - polish all steps
  - auto-fill a single step

What it still does **not** do:

- it does not control your computer
- it does not click through websites/apps
- it does not complete outside tasks on its own

So now Gemini is doing **real workflow content automation inside the guide builder**, especially with screenshots, not just wording cleanup.

The strongest next upgrade would be adding a feature where you can upload **multiple screenshots at once** and Gemini automatically creates the whole guide in one shot, without you making each step first.

Guide-builder-tool

sounds good

I'm turning that into a real one-shot flow: upload multiple screenshots at once, have them turned into step cards automatically, and then let Gemini build the full guide from that image sequence without you creating each step first.

Thought for 22s ›

Edited Guide-builder-tool

Done — I added the **one-shot multi-screenshot flow**.

What it does now:

- you can click **Upload screenshots**
- select **multiple images at once**
- the app automatically creates step cards from them
- then it immediately tries to **build the full guide from that screenshot sequence**

So now Gemini's role is much more concrete:

- **single screenshot** → infer what that step is doing
- **multiple screenshots** → build an ordered guide from the full sequence
- **existing guide content** → fill in title, purpose, materials, notes, and estimated time
- **rough outline text** → convert into steps

That makes it feel more like actual automation inside the builder rather than just rewriting text.

The next best upgrade would be adding **drag-and-drop screenshot ordering** before the one-shot build, so you can rearrange image sequence if Gemini gets the intended flow wrong.

↳ Guide-builder-tool

Go for it

I'm keeping the multi-upload flow, but adding a proper screenshot sequence lane so you can drag thumbnails into the right order before building the guide.

Thought for 22s ›

Guide-builder-tool · typescript Copy Edit Download Preview

```
import React, { useEffect, useMemo, useRef, useState } from "react";
import { motion } from "framer-motion";
import {
  Plus,
  Trash2,
  Copy,
  Download,
  Sparkles,
  RotateCcw,
  GripVertical,
  FileText,
  CheckCircle2,
  Wand2,
  ImagePlus,
  X,
  Bot,
  Zap,
  WandSparkles,
  Upload,
```

```
  Database,
  Save,
  RefreshCw,
  Cloud,
  HardDrive,
  Wifi,
  WifiOff,
  Server,
  FolderOpen,
  AlertCircle,
  Trash,
  KeyRound,
  ShieldAlert,
  Loader2,
  BrainCircuit,
  ScanText,
  Images,
  Clock3,
  Package,
  ClipboardList,
} from "lucide-react";
import { Card, CardContent, CardHeader, CardTitle, CardDescription } fr
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Textarea } from "@/components/ui/textarea";
import { Label } from "@/components/ui/label";
import { Badge } from "@/components/ui/badge";
import { Tabs, TabsContent, TabsList, TabsTrigger } from "@/components/
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue
import { Separator } from "@/components/ui/separator";
import { ScrollArea } from "@/components/ui/scroll-area";

type StepImage = {
  src: string;
  name: string;
  caption: string;
};

type Step = {
  id: string;
  title: string;
  instruction: string;
  tip: string;
  caution: string;
  image: StepImage | null;
};

type GuideData = {
```

```
  title: string;
  audience: string;
  purpose: string;
  estimatedTime: string;
  materials: string;
  notes: string;
  steps: Step[];
};

type SavedGuideRecord = {
  id: string;
  name: string;
  updatedAt: string;
  data: GuideData;
};

type BackendMode = "local" | "remote";
type SyncState = "idle" | "syncing" | "success" | "error";
type ConnectionState = "unknown" | "online" | "offline";
type GeminiStepPayload = Pick<Step, "title" | "instruction" | "tip" | "
type GeminiGuidePayload = {
  title?: string;
  audience?: string;
  purpose?: string;
  estimatedTime?: string;
  materials?: string;
  notes?: string;
  steps?: GeminiStepPayload[];
};

type GeminiVisionStepPayload = {
  title?: string;
  instruction?: string;
  tip?: string;
  caution?: string;
  caption?: string;
};

type GeminiGuideFromImagesPayload = {
  title?: string;
  audience?: string;
  purpose?: string;
  estimatedTime?: string;
  materials?: string;
  notes?: string;
  steps?: Array<GeminiVisionStepPayload & { imageName?: string }>;
};
```

```
const STORAGE_KEY = "guide-builder.saved-guides";
const SETTINGS_KEY = "guide-builder.backend-settings";

const starterSteps: Step[] = [
  {
    id: crypto.randomUUID(),
    title: "Open the task",
    instruction: "Start by opening the system, file, or request you nee
    tip: "Make sure you have the correct case or document before moving
    caution: "Do not proceed if key information is missing.",
    image: null,
  },
  {
    id: crypto.randomUUID(),
    title: "Complete the action",
    instruction: "Follow the required process and enter the needed info
    tip: "Use consistent wording and double-check dates, names, and det
    caution: "Avoid leaving any required fields blank.",
    image: null,
  },
  {
    id: crypto.randomUUID(),
    title: "Review and finalize",
    instruction: "Check your work for accuracy, save it, and send or fi
    tip: "A final review helps catch small mistakes before they become
    caution: "Do not submit until everything has been verified.",
    image: null,
  },
];

const templates: Record<string, GuideData> = {
  blank: {
    title: "",
    audience: "",
    purpose: "",
    estimatedTime: "",
    materials: "",
    notes: "",
    steps: [
      {
        id: crypto.randomUUID(),
        title: "",
        instruction: "",
        tip: "",
        caution: "",
        image: null,
      },
    ],
```

```
    },
    quickstart: {
      title: "Quick Start Guide",
      audience: "New team members",
      purpose: "Use this guide to complete the task quickly and correctly
      estimatedTime: "5–10 minutes",
      materials: "System access, required forms, reference notes",
      notes: "Keep the wording simple and easy to scan.",
      steps: starterSteps,
    },
    sop: {
      title: "Standard Operating Procedure",
      audience: "Operations staff",
      purpose: "This SOP explains the standard process for completing the
      estimatedTime: "10–15 minutes",
      materials: "Access to systems, forms, and any supporting records",
      notes: "Escalate unusual cases to the appropriate reviewer.",
      steps: [
        {
          id: crypto.randomUUID(),
          title: "Prepare",
          instruction: "Gather the required documents and confirm you are
          tip: "Check identifiers first so you do not waste time on the w
          caution: "Missing documents can delay the process.",
          image: null,
        },
        {
          id: crypto.randomUUID(),
          title: "Process",
          instruction: "Complete the required actions in the system and d
          tip: "Use consistent phrasing for notes and communication.",
          caution: "Do not assume missing information—verify it.",
          image: null,
        },
        {
          id: crypto.randomUUID(),
          title: "Quality check",
          instruction: "Review everything before submission to confirm co
          tip: "Compare your final output against the original request.",
          caution: "Errors in names, dates, or attachments can create rew
          image: null,
        },
      ],
    },
    checklist: {
      title: "Checklist Guide",
      audience: "Any team member",
      purpose: "Use this checklist to make sure no required step is misse
```

```
    estimatedTime: "3–7 minutes",
    materials: "Task details and any required references",
    notes: "Ideal for repeatable tasks and quick handoffs.",
    steps: [
      {
        id: crypto.randomUUID(),
        title: "Confirm request details",
        instruction: "Verify the name, date, task type, and any special
        tip: "Read all notes before starting.",
        caution: "Do not assume the last case was the same.",
        image: null,
      },
      {
        id: crypto.randomUUID(),
        title: "Complete required items",
        instruction: "Finish each required item in order and mark it co
        tip: "Work top to bottom to stay organized.",
        caution: "Skipping one field can cause the whole task to come b
        image: null,
      },
      {
        id: crypto.randomUUID(),
        title: "Close out",
        instruction: "Save, send, or file the work and update the final
        tip: "Leave a clean note for the next person if needed.",
        caution: "Do not close until the final status is accurate.",
        image: null,
      },
    ],
  },
};

function createEmptyStep(): Step {
  return {
    id: crypto.randomUUID(),
    title: "",
    instruction: "",
    tip: "",
    caution: "",
    image: null,
  };
}

function cloneTemplate(template: GuideData): GuideData {
  return {
    ...template,
    steps: template.steps.map((s) => ({
      ...s,
```

```
      id: crypto.randomUUID(),
      image: s.image ? { ...s.image } : null,
    })),
  };
}

function cloneGuideData(data: GuideData): GuideData {
  return {
    ...data,
    steps: data.steps.map((s) => ({
      ...s,
      image: s.image ? { ...s.image } : null,
    })),
  };
}

function toTitleCase(value: string) {
  return value
    .trim()
    .toLowerCase()
    .split(" ")
    .filter(Boolean)
    .map((word) => word.charAt(0).toUpperCase() + word.slice(1))
    .join(" ");
}

function sentenceCase(value: string) {
  const trimmed = value.trim();
  if (!trimmed) return "";
  const normalized = trimmed.charAt(0).toUpperCase() + trimmed.slice(1)
  return /[.!?]$/.test(normalized) ? normalized : `${normalized}.`;
}

function generateInstruction(title: string, guideTitle: string, audienc
  const task = title.trim() || "complete this step";
  const guideContext = guideTitle.trim() || "this guide";
  const audienceText = audience.trim() ? ` for ${audience.trim()}` : ""
  return sentenceCase(`Complete ${task.toLowerCase()} as part of ${guid
}

function generateTip(title: string) {
  const task = title.trim() || "this task";
  return sentenceCase(`Keep your wording consistent and double-check ke
}

function generateCaution(title: string) {
  const task = title.trim() || "this step";
  return sentenceCase(`Do not move past ${task.toLowerCase()} until all
```

```
}

function buildStepsFromOutline(outline: string, guide: GuideData): Step
  const items = outline
    .split(/\n|;/)
    .map((line) => line.replace(/^\s*(?:[-*•]|\d+[.)])\s*/, "").trim())
    .filter(Boolean);

  if (items.length === 0) return [];

  return items.map((item) => {
    const title = toTitleCase(item);
    return {
      id: crypto.randomUUID(),
      title,
      instruction: generateInstruction(title, guide.title, guide.audien
      tip: generateTip(title),
      caution: generateCaution(title),
      image: null,
    };
  });
}

function buildMarkdown(guide: GuideData) {
  const lines: string[] = [];
  lines.push(`# ${guide.title || "Untitled Guide"}`);
  lines.push("");
  if (guide.audience) lines.push(`**Audience:** ${guide.audience}`);
  if (guide.purpose) lines.push(`**Purpose:** ${guide.purpose}`);
  if (guide.estimatedTime) lines.push(`**Estimated Time:** ${guide.estir
  if (guide.materials) lines.push(`**Materials Needed:** ${guide.materia
  if (guide.notes) lines.push(`**Notes:** ${guide.notes}`);
  if (guide.audience || guide.purpose || guide.estimatedTime || guide.ma

  lines.push("## Steps");
  lines.push("");
  guide.steps.forEach((step, index) => {
    lines.push(`### ${index + 1}. ${step.title || `Step ${index + 1}`}`
    if (step.instruction) lines.push(step.instruction);
    if (step.tip) lines.push(`- Tip: ${step.tip}`);
    if (step.caution) lines.push(`- Watch out: ${step.caution}`);
    if (step.image?.name) lines.push(`- Image: ${step.image.name}${step
    lines.push("");
  });

  return lines.join("\n");
}
```

```
function readLocalGuides(): SavedGuideRecord[] {
  try {
    const raw = localStorage.getItem(STORAGE_KEY);
    if (!raw) return [];
    const parsed = JSON.parse(raw);
    return Array.isArray(parsed) ? parsed : [];
  } catch {
    return [];
  }
}

function writeLocalGuides(guides: SavedGuideRecord[]) {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(guides));
}

function sortGuides(guides: SavedGuideRecord[]) {
  return [...guides].sort((a, b) => new Date(b.updatedAt).getTime() - n
}

function normalizeGuideList(payload: unknown): SavedGuideRecord[] {
  if (Array.isArray(payload)) return payload as SavedGuideRecord[];
  if (payload && typeof payload === "object" && Array.isArray((payload
    return (payload as { guides: SavedGuideRecord[] }).guides;
  }
  return [];
}

async function fetchRemoteGuides(apiBaseUrl: string): Promise<SavedGuid
  const response = await fetch(`${apiBaseUrl.replace(/\/$/, "")}/api/gu
    method: "GET",
    headers: { "Content-Type": "application/json" },
  });
  if (!response.ok) {
    throw new Error(`Unable to load guides (${response.status})`);
  }
  const payload = await response.json();
  return sortGuides(normalizeGuideList(payload));
}

async function saveRemoteGuide(apiBaseUrl: string, record: SavedGuideRe
  const response = await fetch(`${apiBaseUrl.replace(/\/$/, "")}/api/gu
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(record),
  });
  if (!response.ok) {
    throw new Error(`Unable to save guide (${response.status})`);
  }
```

```
  return response.json().catch(() => record);
}

async function deleteRemoteGuide(apiBaseUrl: string, id: string) {
  const response = await fetch(`${apiBaseUrl.replace(/\/$/, "")}/api/gu
    method: "DELETE",
    headers: { "Content-Type": "application/json" },
  });
  if (!response.ok) {
    throw new Error(`Unable to delete guide (${response.status})`);
  }
}

async function testRemoteConnection(apiBaseUrl: string) {
  const cleanBase = apiBaseUrl.replace(/\/$/, "");
  const urls = [`${cleanBase}/api/health`, `${cleanBase}/api/guides`];

  for (const url of urls) {
    try {
      const response = await fetch(url, { method: "GET" });
      if (response.ok) return true;
    } catch {
      // try next endpoint
    }
  }

  throw new Error("Connection test failed");
}

function stripCodeFences(value: string) {
  return value.replace(/^```(?:json)?\s*/i, "").replace(/```$/i, "").tr
}

function parseJsonFromText<T>(value: string): T {
  const clean = stripCodeFences(value);
  try {
    return JSON.parse(clean) as T;
  } catch {
    const firstBrace = clean.indexOf("{");
    const lastBrace = clean.lastIndexOf("}");
    if (firstBrace !== -1 && lastBrace !== -1 && lastBrace > firstBrace
      return JSON.parse(clean.slice(firstBrace, lastBrace + 1)) as T;
    }
    throw new Error("Gemini returned invalid JSON.");
  }
}

function extractDataUrlParts(dataUrl: string) {
```

```
  const match = dataUrl.match(/^data:(.*?);base64,(.*)$/);
  if (!match) return null;
  return { mimeType: match[1], data: match[2] };
}

async function fileToStepImage(file: File): Promise<StepImage> {
  return await new Promise((resolve, reject) => {
    const reader = new FileReader();
    reader.onload = () => {
      resolve({
        src: String(reader.result || ""),
        name: file.name,
        caption: "",
      });
    };
    reader.onerror = () => reject(new Error(`Could not read ${file.name
    reader.readAsDataURL(file);
  });
}

function normalizeAiGuidePayload(payload: GeminiGuidePayload, currentGu
  const fallbackSteps = currentGuide.steps.length > 0 ? currentGuide.st
  const nextStepsSource = Array.isArray(payload.steps) && payload.steps

  return {
    title: payload.title?.trim() || currentGuide.title,
    audience: payload.audience?.trim() || currentGuide.audience,
    purpose: payload.purpose?.trim() || currentGuide.purpose,
    estimatedTime: payload.estimatedTime?.trim() || currentGuide.estima
    materials: payload.materials?.trim() || currentGuide.materials,
    notes: payload.notes?.trim() || currentGuide.notes,
    steps: nextStepsSource.map((step, index) => {
      const existingImage = currentGuide.steps[index]?.image ?? null;
      return {
        id: crypto.randomUUID(),
        title: step.title?.trim() || currentGuide.steps[index]?.title |
        instruction: step.instruction?.trim() || currentGuide.steps[ind
        tip: step.tip?.trim() || currentGuide.steps[index]?.tip || "",
        caution: step.caution?.trim() || currentGuide.steps[index]?.cau
        image: existingImage,
      };
    }),
  };
}

const glass =
  "border border-white/20 bg-white/10 backdrop-blur-2xl shadow-[0_20px_
```

```
const glassSoft =
  "border border-white/25 bg-white/8 backdrop-blur-xl shadow-[0_12px_36|

export default function GuideBuilderTool() {
  const [selectedTemplate, setSelectedTemplate] = useState("quickstart"
  const [guide, setGuide] = useState<GuideData>(cloneTemplate(templates
  const [copied, setCopied] = useState(false);
  const [activeTab, setActiveTab] = useState("builder");
  const [outlineInput, setOutlineInput] = useState("");
  const [automationMode, setAutomationMode] = useState<"replace" | "app

  const [backendMode, setBackendMode] = useState<BackendMode>("local");
  const [apiBaseUrl, setApiBaseUrl] = useState("http://localhost:3001")
  const [savedGuides, setSavedGuides] = useState<SavedGuideRecord[]>([]
  const [currentGuideId, setCurrentGuideId] = useState<string>(crypto.r
  const [syncState, setSyncState] = useState<SyncState>("idle");
  const [syncMessage, setSyncMessage] = useState("Not synced yet.");
  const [connectionState, setConnectionState] = useState<ConnectionStat
  const [isTestingConnection, setIsTestingConnection] = useState(false)
  const [isLoadingGuides, setIsLoadingGuides] = useState(false);

  const [geminiApiKey, setGeminiApiKey] = useState("");
  const [showGeminiKey, setShowGeminiKey] = useState(false);
  const [geminiModel, setGeminiModel] = useState("gemini-2.5-flash");
  const [useGeminiAutomation, setUseGeminiAutomation] = useState(false)
  const [isAiWorking, setIsAiWorking] = useState(false);
  const [aiStatusMessage, setAiStatusMessage] = useState("Gemini automa
  const [draggedStepId, setDraggedStepId] = useState<string | null>(nul

  const bulkImageInputRef = useRef<HTMLInputElement | null>(null);

  const markdown = useMemo(() => buildMarkdown(guide), [guide]);
  const currentGuideName = guide.title.trim() || "Untitled Guide";
  const isGeminiReady = geminiApiKey.trim().length > 0;
  const stepsWithPhotos = guide.steps.filter((s) => s.image?.src).lengt
  const filledSteps = guide.steps.filter((s) => s.title || s.instructio
  const imageSteps = guide.steps.filter((step) => step.image?.src);

  useEffect(() => {
    try {
      const raw = localStorage.getItem(SETTINGS_KEY);
      if (!raw) {
        setSavedGuides(sortGuides(readLocalGuides()));
        return;
      }
      const parsed = JSON.parse(raw);
      if (parsed?.backendMode === "local" || parsed?.backendMode === "r
        setBackendMode(parsed.backendMode);
```

```
        }
        if (typeof parsed?.apiBaseUrl === "string" && parsed.apiBaseUrl.t
          setApiBaseUrl(parsed.apiBaseUrl);
        }
        if (typeof parsed?.geminiApiKey === "string") {
          setGeminiApiKey(parsed.geminiApiKey);
        }
        if (typeof parsed?.geminiModel === "string" && parsed.geminiModel
          setGeminiModel(parsed.geminiModel);
        }
        if (typeof parsed?.useGeminiAutomation === "boolean") {
          setUseGeminiAutomation(parsed.useGeminiAutomation);
        }
        setSavedGuides(sortGuides(readLocalGuides()));
      } catch {
        setSavedGuides(sortGuides(readLocalGuides()));
      }
    }, []);

    useEffect(() => {
      localStorage.setItem(
        SETTINGS_KEY,
        JSON.stringify({
          backendMode,
          apiBaseUrl,
          geminiApiKey,
          geminiModel,
          useGeminiAutomation,
        })
      );
    }, [backendMode, apiBaseUrl, geminiApiKey, geminiModel, useGeminiAuto

    useEffect(() => {
      void refreshSavedGuides();
    }, [backendMode]);

    const updateGuide = (field: keyof GuideData, value: string) => {
      setGuide((prev) => ({ ...prev, [field]: value }));
    };

    const updateStep = (id: string, field: keyof Step, value: string | St
      setGuide((prev) => ({
        ...prev,
        steps: prev.steps.map((step) => (step.id === id ? { ...step, [fie
      }));
    };

    const addStep = () => {
```

```
    setGuide((prev) => ({
      ...prev,
      steps: [...prev.steps, createEmptyStep()],
    }));
  };

  const deleteStep = (id: string) => {
    setGuide((prev) => ({
      ...prev,
      steps: prev.steps.length > 1 ? prev.steps.filter((step) => step.i
    }));
  };

  const moveStep = (index: number, direction: "up" | "down") => {
    setGuide((prev) => {
      const steps = [...prev.steps];
      const newIndex = direction === "up" ? index - 1 : index + 1;
      if (newIndex < 0 || newIndex >= steps.length) return prev;
      [steps[index], steps[newIndex]] = [steps[newIndex], steps[index]]
      return { ...prev, steps };
    });
  };

  const moveStepById = (sourceId: string, targetId: string) => {
    if (!sourceId || !targetId || sourceId === targetId) return;

    setGuide((prev) => {
      const steps = [...prev.steps];
      const sourceIndex = steps.findIndex((step) => step.id === sourceI
      const targetIndex = steps.findIndex((step) => step.id === targetI
      if (sourceIndex === -1 || targetIndex === -1) return prev;

      const [moved] = steps.splice(sourceIndex, 1);
      steps.splice(targetIndex, 0, moved);
      return { ...prev, steps };
    });
  };

  const handleSequenceDragStart = (stepId: string) => {
    setDraggedStepId(stepId);
  };

  const handleSequenceDrop = (targetStepId: string) => {
    if (!draggedStepId) return;
    moveStepById(draggedStepId, targetStepId);
    setDraggedStepId(null);
    setAiStatusMessage("Reordered the screenshot sequence. Build the gu
  };
```

```
const applyTemplate = (key: string) => {
  setSelectedTemplate(key);
  setCurrentGuideId(crypto.randomUUID());
  setGuide(cloneTemplate(templates[key]));
  setSyncState("idle");
  setSyncMessage("Template loaded. Save when ready.");
};

const resetGuide = () => {
  applyTemplate(selectedTemplate);
  setOutlineInput("");
};

const copyMarkdown = async () => {
  await navigator.clipboard.writeText(markdown);
  setCopied(true);
  setTimeout(() => setCopied(false), 1800);
};

const downloadMarkdown = () => {
  const blob = new Blob([markdown], { type: "text/markdown;charset=ut
  const url = URL.createObjectURL(blob);
  const link = document.createElement("a");
  link.href = url;
  link.download = `${currentGuideName.toLowerCase().replace(/[^a-z0-9
  link.click();
  URL.revokeObjectURL(url);
};

const runLocalOutlineGeneration = () => {
  const generatedSteps = buildStepsFromOutline(outlineInput, guide);
  if (generatedSteps.length === 0) return false;

  setGuide((prev) => ({
    ...prev,
    steps: automationMode === "replace" ? generatedSteps : [...prev.st
  }));
  return true;
};

const runLocalPolishAll = () => {
  setGuide((prev) => ({
    ...prev,
    steps: prev.steps.map((step, index) => {
      const title = step.title.trim() || `Step ${index + 1}`;
      return {
        ...step,
```

```
            title: toTitleCase(title),
            instruction: step.instruction.trim() || generateInstruction(t
            tip: step.tip.trim() || generateTip(title),
            caution: step.caution.trim() || generateCaution(title),
          };
        }),
      }));
    };

    const runLocalSingleStepFill = (step: Step, index: number) => {
      const title = step.title.trim() || `Step ${index + 1}`;
      updateStep(step.id, "title", toTitleCase(title));
      updateStep(step.id, "instruction", step.instruction.trim() || gener
      updateStep(step.id, "tip", step.tip.trim() || generateTip(title));
      updateStep(step.id, "caution", step.caution.trim() || generateCauti
    };

    const callGeminiJson = async <T,>(prompt: string): Promise<T> => {
      if (!geminiApiKey.trim()) {
        throw new Error("Add your Gemini API key first.");
      }

      const response = await fetch(
        `https://generativelanguage.googleapis.com/v1beta/models/${encodel
        {
          method: "POST",
          headers: { "Content-Type": "application/json" },
          body: JSON.stringify({
            contents: [
              {
                role: "user",
                parts: [{ text: prompt }],
              },
            ],
            generationConfig: {
              temperature: 0.25,
              responseMimeType: "application/json",
            },
          }),
        }
      );

      if (!response.ok) {
        throw new Error(`Gemini request failed (${response.status}).`);
      }

      const payload = await response.json();
      const text = (payload?.candidates?.[0]?.content?.parts ?? [])
```

```
        .map((part: { text?: string }) => part.text || "")
        .join("\n")
        .trim();

      if (!text) {
        throw new Error("Gemini returned an empty response.");
      }

      return parseJsonFromText<T>(text);
    };

    const callGeminiWithParts = async <T,>(parts: Array<{ text?: string;
      if (!geminiApiKey.trim()) {
        throw new Error("Add your Gemini API key first.");
      }

      const response = await fetch(
        `https://generativelanguage.googleapis.com/v1beta/models/${encodel
        {
          method: "POST",
          headers: { "Content-Type": "application/json" },
          body: JSON.stringify({
            contents: [
              {
                role: "user",
                parts,
              },
            ],
            generationConfig: {
              temperature: 0.2,
              responseMimeType: "application/json",
            },
          }),
        }
      );

      if (!response.ok) {
        throw new Error(`Gemini request failed (${response.status}).`);
      }

      const payload = await response.json();
      const text = (payload?.candidates?.[0]?.content?.parts ?? [])
        .map((part: { text?: string }) => part.text || "")
        .join("\n")
        .trim();

      if (!text) {
        throw new Error("Gemini returned an empty response.");
```

```
    }

    return parseJsonFromText<T>(text);
  };

  const buildGuideFromImageSteps = async (imageSteps: Step[]) => {
    if (imageSteps.length === 0) {
      setAiStatusMessage("Attach at least one image to a step first.");
      return;
    }

    if (!useGeminiAutomation || !isGeminiReady) {
      const rebuiltSteps = imageSteps.map((step, index) => ({
        id: crypto.randomUUID(),
        title: step.title || `Step ${index + 1}`,
        instruction: step.instruction || `Review the attached screensho
        tip: step.tip || `Use the screenshot as a visual reference whil
        caution: step.caution || `Confirm that the screen shown matches
        image: step.image,
      }));
      setGuide((prev) => ({
        ...prev,
        steps: rebuiltSteps,
        title: prev.title || "Guide From Screenshots",
        purpose: prev.purpose || "Use this guide to follow the screensh
      }));
      setAiStatusMessage("Built a simple image-based guide using the lo
      return;
    }

    const parts: Array<{ text?: string; inlineData?: { mimeType: string
      {
        text: [
          "You are building a step-by-step internal guide from a series
          "Return only valid JSON.",
          "Infer the likely workflow order from the image sequence prov
          "Generate a concise title, purpose, estimatedTime, materials,
          "Each step should explain what to do on or with that screen."
          "Use this exact JSON shape:",
          JSON.stringify({
            title: "",
            audience: "",
            purpose: "",
            estimatedTime: "",
            materials: "",
            notes: "",
            steps: [{ imageName: "", title: "", instruction: "", tip: "
          }),
```

```
        `Current guide title: ${guide.title || ""}`,
        `Current audience: ${guide.audience || ""}`,
        `Current purpose: ${guide.purpose || ""}`,
        `There are ${imageSteps.length} images in this sequence. Keep
      ].join("\n\n"),
    },
  ];

  imageSteps.forEach((step, index) => {
    const partsFromImage = step.image?.src ? extractDataUrlParts(step
    if (!partsFromImage) return;
    parts.push({
      text: `Image ${index + 1}: ${step.image?.name || `step-${index +
    });
    parts.push({
      inlineData: {
        mimeType: partsFromImage.mimeType,
        data: partsFromImage.data,
      },
    });
  });

  setIsAiWorking(true);
  setAiStatusMessage("Gemini is building a guide from the attached ima

  try {
    const result = await callGeminiWithParts<GeminiGuideFromImagesPay
    const nextSteps = (result.steps || []).map((item, index) => ({
      id: crypto.randomUUID(),
      title: item.title?.trim() || `Step ${index + 1}`,
      instruction: item.instruction?.trim() || "",
      tip: item.tip?.trim() || "",
      caution: item.caution?.trim() || "",
      image: {
        ...(imageSteps[index].image as StepImage),
        caption: item.caption?.trim() || imageSteps[index].image?.cap
      },
    }));

    setGuide((prev) => ({
      ...prev,
      title: result.title?.trim() || prev.title || "Guide From Screen
      audience: result.audience?.trim() || prev.audience,
      purpose: result.purpose?.trim() || prev.purpose,
      estimatedTime: result.estimatedTime?.trim() || prev.estimatedTi
      materials: result.materials?.trim() || prev.materials,
      notes: result.notes?.trim() || prev.notes,
      steps: nextSteps.length > 0 ? nextSteps : prev.steps,
```

```
      }));
      setAiStatusMessage("Gemini built the guide from the attached imag
    } catch (error) {
      setAiStatusMessage(error instanceof Error ? error.message : "Gemi
    } finally {
      setIsAiWorking(false);
    }
  };

  const autoGenerateFromOutline = async () => {
    if (!outlineInput.trim()) return;

    if (!useGeminiAutomation || !isGeminiReady) {
      const didGenerate = runLocalOutlineGeneration();
      if (didGenerate) {
        setAiStatusMessage("Generated steps using the built-in local au
      }
      return;
    }

    setIsAiWorking(true);
    setAiStatusMessage("Gemini is generating your steps...");

    try {
      const prompt = [
        "You are helping create a clean internal guide.",
        "Return only valid JSON.",
        "Create concise but useful guide automation output from the rou
        "Preserve intent, avoid fluff, and write in clear workplace Eng
        "Use this exact JSON shape:",
        JSON.stringify({
          title: "",
          audience: "",
          purpose: "",
          estimatedTime: "",
          materials: "",
          notes: "",
          steps: [{ title: "", instruction: "", tip: "", caution: "" }]
        }),
        `Current guide title: ${guide.title || ""}`,
        `Current audience: ${guide.audience || ""}`,
        `Current purpose: ${guide.purpose || ""}`,
        `Generation mode: ${automationMode}`,
        `Rough outline:\n${outlineInput}`,
      ].join("\n\n");

      const result = await callGeminiJson<GeminiGuidePayload>(prompt);
      const aiGuide = normalizeAiGuidePayload(result, guide);
```

```
      setGuide((prev) => ({
        ...prev,
        title: aiGuide.title || prev.title,
        audience: aiGuide.audience || prev.audience,
        purpose: aiGuide.purpose || prev.purpose,
        estimatedTime: aiGuide.estimatedTime || prev.estimatedTime,
        materials: aiGuide.materials || prev.materials,
        notes: aiGuide.notes || prev.notes,
        steps: automationMode === "replace" ? aiGuide.steps : [...prev.s
      }));
      setAiStatusMessage("Gemini generated the guide steps successfully
    } catch (error) {
      const didGenerate = runLocalOutlineGeneration();
      setAiStatusMessage(
        `${error instanceof Error ? error.message : "Gemini generation f
      );
    } finally {
      setIsAiWorking(false);
    }
  };

  const autoPolishSteps = async () => {
    if (!useGeminiAutomation || !isGeminiReady) {
      runLocalPolishAll();
      setAiStatusMessage("Polished all steps using the built-in local au
      return;
    }

    setIsAiWorking(true);
    setAiStatusMessage("Gemini is polishing the guide...");

    try {
      const prompt = [
        "You are polishing an internal step-by-step guide.",
        "Return only valid JSON.",
        "Improve clarity, consistency, and brevity, but do not add fluff
        "Preserve meaning, keep workplace tone, and keep each step actio
        "Use this exact JSON shape:",
        JSON.stringify({
          title: "",
          audience: "",
          purpose: "",
          estimatedTime: "",
          materials: "",
          notes: "",
          steps: [{ title: "", instruction: "", tip: "", caution: "" }]
        }),
```

```
          `Guide JSON:\n${JSON.stringify({
            title: guide.title,
            audience: guide.audience,
            purpose: guide.purpose,
            estimatedTime: guide.estimatedTime,
            materials: guide.materials,
            notes: guide.notes,
            steps: guide.steps.map((step) => ({
              title: step.title,
              instruction: step.instruction,
              tip: step.tip,
              caution: step.caution,
            })),
          })}`,
        ].join("\n\n");

        const result = await callGeminiJson<GeminiGuidePayload>(prompt);
        setGuide(normalizeAiGuidePayload(result, guide));
        setAiStatusMessage("Gemini polished the guide successfully.");
      } catch (error) {
        runLocalPolishAll();
        setAiStatusMessage(`${error instanceof Error ? error.message : "G
      } finally {
        setIsAiWorking(false);
      }
    };

    const autoFillSingleStep = async (step: Step, index: number) => {
      if (!useGeminiAutomation || !isGeminiReady) {
        runLocalSingleStepFill(step, index);
        setAiStatusMessage("Filled the step using the built-in local auto
        return;
      }

      setIsAiWorking(true);
      setAiStatusMessage(`Gemini is improving step ${index + 1}...`);

      try {
        const prompt = [
          "You are improving one step in an internal guide.",
          "Return only valid JSON.",
          "Keep the step concise, clear, and actionable.",
          "Use this exact JSON shape:",
          JSON.stringify({ title: "", instruction: "", tip: "", caution: "
          `Guide title: ${guide.title}`,
          `Guide audience: ${guide.audience}`,
          `Guide purpose: ${guide.purpose}`,
          `Current step JSON: ${JSON.stringify({
```

```
            title: step.title,
            instruction: step.instruction,
            tip: step.tip,
            caution: step.caution,
          })}`,
        ].join("\n\n");

        const result = await callGeminiJson<GeminiStepPayload>(prompt);
        updateStep(step.id, "title", result.title?.trim() || step.title |
        updateStep(step.id, "instruction", result.instruction?.trim() || s
        updateStep(step.id, "tip", result.tip?.trim() || step.tip || gene
        updateStep(step.id, "caution", result.caution?.trim() || step.cau
        setAiStatusMessage(`Gemini improved step ${index + 1}.`);
      } catch (error) {
        runLocalSingleStepFill(step, index);
        setAiStatusMessage(`${error instanceof Error ? error.message : "G
      } finally {
        setIsAiWorking(false);
      }
    };

    const autoBuildGuideMeta = async () => {
      const inputContext = [
        guide.title && `Title: ${guide.title}`,
        guide.audience && `Audience: ${guide.audience}`,
        guide.purpose && `Purpose: ${guide.purpose}`,
        guide.notes && `Notes: ${guide.notes}`,
        outlineInput && `Outline: ${outlineInput}`,
        guide.steps.length > 0
          ? `Steps:\n${guide.steps
              .map((step, index) => `${index + 1}. ${step.title || "Untit
              .join("\n")}`
          : "",
      ]
        .filter(Boolean)
        .join("\n\n");

      if (!inputContext.trim()) {
        setAiStatusMessage("Add some guide content first so the automatio
        return;
      }

      if (!useGeminiAutomation || !isGeminiReady) {
        const fallbackTitle = guide.title || "Process Guide";
        const fallbackPurpose = guide.purpose || sentenceCase(`Use this gu
        const fallbackMaterials = guide.materials || "Relevant system acc
        const fallbackTime = guide.estimatedTime || `${Math.max(5, guide.s
        setGuide((prev) => ({
```

```
        ...prev,
        purpose: fallbackPurpose,
        materials: fallbackMaterials,
        estimatedTime: fallbackTime,
      }));
      setAiStatusMessage("Filled guide details using the built-in local
      return;
    }

    setIsAiWorking(true);
    setAiStatusMessage("Gemini is generating guide details...");

    try {
      const prompt = [
        "You are filling in missing guide metadata for an internal step
        "Return only valid JSON.",
        "Use concise workplace wording.",
        "Prioritize title, purpose, estimatedTime, materials, and notes
        "Use this exact JSON shape:",
        JSON.stringify({
          title: "",
          audience: "",
          purpose: "",
          estimatedTime: "",
          materials: "",
          notes: "",
          steps: [],
        }),
        `Guide context:\n${inputContext}`,
      ].join("\n\n");

      const result = await callGeminiJson<GeminiGuidePayload>(prompt);
      setGuide((prev) => ({
        ...prev,
        title: result.title?.trim() || prev.title,
        audience: result.audience?.trim() || prev.audience,
        purpose: result.purpose?.trim() || prev.purpose,
        estimatedTime: result.estimatedTime?.trim() || prev.estimatedTi
        materials: result.materials?.trim() || prev.materials,
        notes: result.notes?.trim() || prev.notes,
      }));
      setAiStatusMessage("Gemini filled the guide details successfully.'
    } catch (error) {
      setAiStatusMessage(error instanceof Error ? error.message : "Gemi
    } finally {
      setIsAiWorking(false);
    }
  };
```

```
const autoAnalyzeStepImage = async (step: Step, index: number) => {
  if (!step.image?.src) {
    setAiStatusMessage("Attach an image to this step first.");
    return;
  }

  if (!useGeminiAutomation || !isGeminiReady) {
    const fallbackTitle = step.title || `Review screen ${index + 1}`;
    updateStep(step.id, "title", fallbackTitle);
    updateStep(step.id, "instruction", step.instruction || `Review th
    updateStep(step.id, "tip", step.tip || `Use the screenshot as a v
    updateStep(step.id, "caution", step.caution || `Double-check that
    updateStep(step.id, "image", { ...step.image, caption: step.image
    setAiStatusMessage("Filled the step from the image using the buil
    return;
  }

  const imageParts = extractDataUrlParts(step.image.src);
  if (!imageParts) {
    setAiStatusMessage("That image format could not be read for Gemin
    return;
  }

  setIsAiWorking(true);
  setAiStatusMessage(`Gemini is analyzing the image for step ${index +

  try {
    const result = await callGeminiWithParts<GeminiVisionStepPayload>
      {
        text: [
          "You are analyzing a screenshot or reference image for one s
          "Return only valid JSON.",
          "Describe what the user likely needs to do on this screen o
          "Keep it concise, practical, and workplace-friendly.",
          "Use this exact JSON shape:",
          JSON.stringify({ title: "", instruction: "", tip: "", cauti
          `Guide title: ${guide.title || ""}`,
          `Guide audience: ${guide.audience || ""}`,
          `Current step title: ${step.title || ""}`,
          `Current step instruction: ${step.instruction || ""}`,
          `Image filename: ${step.image.name}`,
        ].join("\n\n"),
      },
      {
        inlineData: {
          mimeType: imageParts.mimeType,
          data: imageParts.data,
```

```
            },
          },
        ]);

        updateStep(step.id, "title", result.title?.trim() || step.title |
        updateStep(step.id, "instruction", result.instruction?.trim() || s
        updateStep(step.id, "tip", result.tip?.trim() || step.tip || gene
        updateStep(step.id, "caution", result.caution?.trim() || step.cau
        updateStep(step.id, "image", {
          ...step.image,
          caption: result.caption?.trim() || step.image.caption || step.i
        });
        setAiStatusMessage(`Gemini analyzed the image and filled step ${i
      } catch (error) {
        setAiStatusMessage(error instanceof Error ? error.message : "Gemi
      } finally {
        setIsAiWorking(false);
      }
    };

    const autoBuildGuideFromImages = async () => {
      await buildGuideFromImageSteps(imageSteps);
    };

    const handleBulkScreenshotsUpload = async (files: FileList | null) =>
      if (!files || files.length === 0) return;

      const imageFiles = Array.from(files).filter((file) => file.type.sta
      if (imageFiles.length === 0) {
        setAiStatusMessage("Please upload image files only.");
        return;
      }

      setIsAiWorking(true);
      setAiStatusMessage(`Preparing ${imageFiles.length} screenshot${imag

      try {
        const stepImages = await Promise.all(imageFiles.map((file) => fil
        const imageSteps: Step[] = stepImages.map((image, index) => ({
          id: crypto.randomUUID(),
          title: `Step ${index + 1}`,
          instruction: "",
          tip: "",
          caution: "",
          image,
        }));

        setGuide((prev) => ({
```

```
        ...prev,
        steps: imageSteps,
      }));

      setIsAiWorking(false);
      setAiStatusMessage(`Uploaded ${imageFiles.length} screenshot${imag
    } catch (error) {
      setIsAiWorking(false);
      setAiStatusMessage(error instanceof Error ? error.message : "Coul
    }
  };

  const testGeminiConnection = async () => {
    if (!isGeminiReady) {
      setAiStatusMessage("Add your Gemini API key first.");
      return;
    }

    setIsAiWorking(true);
    setAiStatusMessage("Testing Gemini connection...");

    try {
      await callGeminiJson<{ ok: boolean; message: string }>([
        "Return only valid JSON.",
        "Use this exact JSON shape:",
        JSON.stringify({ ok: true, message: "connected" }),
        "Set ok to true and message to a short connection confirmation.'
      ].join("\n\n"));
      setAiStatusMessage("Gemini connection looks good.");
    } catch (error) {
      setAiStatusMessage(error instanceof Error ? error.message : "Gemi
    } finally {
      setIsAiWorking(false);
    }
  };

  const onStepImageUpload = (stepId: string, file?: File | null) => {
    if (!file) return;
    const reader = new FileReader();
    reader.onload = () => {
      updateStep(stepId, "image", {
        src: String(reader.result || ""),
        name: file.name,
        caption: "",
      });
    };
    reader.readAsDataURL(file);
  };
```

```
const buildCurrentRecord = (): SavedGuideRecord => ({
  id: currentGuideId,
  name: currentGuideName,
  updatedAt: new Date().toISOString(),
  data: cloneGuideData(guide),
});

const refreshSavedGuides = async () => {
  setIsLoadingGuides(true);
  try {
    if (backendMode === "local") {
      setSavedGuides(sortGuides(readLocalGuides()));
      setConnectionState("online");
      return;
    }

    const guides = await fetchRemoteGuides(apiBaseUrl);
    setSavedGuides(guides);
    setConnectionState("online");
  } catch (error) {
    setConnectionState("offline");
    setSavedGuides(sortGuides(readLocalGuides()));
    setSyncState("error");
    setSyncMessage(error instanceof Error ? error.message : "Unable t
  } finally {
    setIsLoadingGuides(false);
  }
};

const saveCurrentGuide = async () => {
  const record = buildCurrentRecord();
  setSyncState("syncing");
  setSyncMessage(backendMode === "remote" ? "Saving to backend..." : '

  try {
    if (backendMode === "local") {
      const existing = readLocalGuides().filter((item) => item.id !==
      const next = sortGuides([record, ...existing]);
      writeLocalGuides(next);
      setSavedGuides(next);
      setConnectionState("online");
    } else {
      await saveRemoteGuide(apiBaseUrl, record);
      setConnectionState("online");
      const refreshed = await fetchRemoteGuides(apiBaseUrl);
      setSavedGuides(refreshed);
    }
```

```
      setSyncState("success");
      setSyncMessage(`Saved ${record.name}.`);
    } catch (error) {
      setConnectionState("offline");
      setSyncState("error");
      setSyncMessage(error instanceof Error ? error.message : "Save fai
    }
  };

  const loadSavedGuide = (record: SavedGuideRecord) => {
    setCurrentGuideId(record.id);
    setGuide(cloneGuideData(record.data));
    setSelectedTemplate("blank");
    setSyncState("idle");
    setSyncMessage(`Loaded ${record.name}.`);
  };

  const deleteSavedGuide = async (id: string) => {
    setSyncState("syncing");
    setSyncMessage("Deleting guide...");

    try {
      if (backendMode === "local") {
        const next = readLocalGuides().filter((item) => item.id !== id)
        writeLocalGuides(next);
        setSavedGuides(sortGuides(next));
        setConnectionState("online");
      } else {
        await deleteRemoteGuide(apiBaseUrl, id);
        const refreshed = await fetchRemoteGuides(apiBaseUrl);
        setSavedGuides(refreshed);
        setConnectionState("online");
      }

      if (currentGuideId === id) {
        setCurrentGuideId(crypto.randomUUID());
      }

      setSyncState("success");
      setSyncMessage("Guide deleted.");
    } catch (error) {
      setConnectionState("offline");
      setSyncState("error");
      setSyncMessage(error instanceof Error ? error.message : "Delete fa
    }
  };
```

```
const runConnectionTest = async () => {
  if (backendMode === "local") {
    setConnectionState("online");
    setSyncState("success");
    setSyncMessage("Local storage is available.");
    return;
  }

  setIsTestingConnection(true);
  setSyncState("syncing");
  setSyncMessage("Testing backend connection...");

  try {
    await testRemoteConnection(apiBaseUrl);
    setConnectionState("online");
    setSyncState("success");
    setSyncMessage("Backend connection looks good.");
  } catch (error) {
    setConnectionState("offline");
    setSyncState("error");
    setSyncMessage(error instanceof Error ? error.message : "Connecti
  } finally {
    setIsTestingConnection(false);
  }
};

return (
  <div className="relative min-h-screen overflow-hidden bg-[radial-gra
    <div className="pointer-events-none absolute inset-0 overflow-hid
      <div className="absolute -left-24 top-10 h-80 w-80 rounded-full
      <div className="absolute right-0 top-0 h-[28rem] w-[28rem] roun
      <div className="absolute bottom-0 left-1/3 h-72 w-72 rounded-fu
      <div className="absolute bottom-10 right-20 h-56 w-56 rounded-f
      <div className="absolute left-[18%] top-[24%] h-64 w-64 rounded
      <div className="absolute right-[18%] top-[34%] h-72 w-72 rounde
      <div className="absolute bottom-[18%] left-[42%] h-60 w-60 roun
      <div className="absolute left-[6%] top-[52%] h-72 w-72 rounded-
      <div className="absolute right-[8%] top-[58%] h-80 w-80 rounded
      <div className="absolute left-[24%] bottom-[10%] h-72 w-72 roun
      <div className="absolute right-[28%] bottom-[6%] h-80 w-80 roun
      <div className="absolute inset-0 bg-[radial-gradient(circle_at_
      <div className="absolute inset-0 bg-[linear-gradient(to_bottom,
    </div>

    <div className="relative mx-auto max-w-7xl p-4 md:p-6">
      <motion.div
        initial={{ opacity: 0, y: 10 }}
        animate={{ opacity: 1, y: 0 }}
```

```
            transition={{ duration: 0.35 }}
            className="mb-6 grid gap-4 md:grid-cols-[1.35fr_0.9fr]"
          >
            <Card className={`${glass} rounded-[28px]`}>
              <CardHeader className="pb-4">
                <div className="flex flex-wrap items-center gap-4">
                  <div className="flex h-14 w-14 items-center justify-cen
                    <FileText className="h-6 w-6" />
                  </div>
                  <div className="min-w-0 flex-1">
                    <div className="mb-2 flex items-center gap-2">
                      <Badge className="rounded-full border border-white/
                        macOS Tahoe Glass
                      </Badge>
                      <Badge className="rounded-full border border-white/
                        Backend-ready
                      </Badge>
                      <Badge className="rounded-full border border-emeral
                        Gemini-ready
                      </Badge>
                    </div>
                    <CardTitle className="text-3xl font-semibold tracking
                      Fast Guide Builder
                    </CardTitle>
                    <CardDescription className="mt-2 max-w-2xl text-[15px
                      Build polished guides quickly with reusable templat
                    </CardDescription>
                  </div>
                </div>
              </CardHeader>
              <CardContent>
                <div className="grid gap-3 sm:grid-cols-4">
                  {[
                    { label: "Template", value: selectedTemplate.replace(
                    { label: "Steps", value: String(guide.steps.length) }
                    { label: "Completed step cards", value: String(filled
                    { label: "Steps with photos", value: String(stepsWith
                  ].map((item) => (
                    <div
                      key={item.label}
                      className="rounded-[24px] border border-white/30 bg
                    >
                      <div className="text-sm text-slate-600">{item.label
                      <div className="mt-1 truncate text-2xl font-semibol
                    </div>
                  ))}
                </div>
              </CardContent>
```

```
            </Card>

            <Card className={`${glass} rounded-[28px]`}>
              <CardHeader>
                <div className="flex items-center gap-3">
                  <div className="flex h-10 w-10 items-center justify-cen
                    <Wand2 className="h-5 w-5 text-slate-800" />
                  </div>
                  <div>
                    <CardTitle className="text-lg text-slate-900">Quick a
                    <CardDescription className="text-slate-700">Start wit
                  </div>
                </div>
              </CardHeader>
              <CardContent className="space-y-3">
                <div className="space-y-2">
                  <Label className="text-slate-800">Choose a template</La
                  <Select value={selectedTemplate} onValueChange={applyTe
                    <SelectTrigger className="rounded-2xl border-white/30
                      <SelectValue placeholder="Select template" />
                    </SelectTrigger>
                    <SelectContent>
                      <SelectItem value="blank">Blank</SelectItem>
                      <SelectItem value="quickstart">Quick Start Guide</S
                      <SelectItem value="sop">Standard Operating Procedur
                      <SelectItem value="checklist">Checklist Guide</Sele
                    </SelectContent>
                  </Select>
                </div>
                <div className="grid gap-2 sm:grid-cols-2">
                  <Button className="rounded-2xl border border-white/35 b
                    <Save className="mr-2 h-4 w-4" />
                    Save guide
                  </Button>
                  <Button className="rounded-2xl border border-white/35 b
                    <Copy className="mr-2 h-4 w-4" />
                    {copied ? "Copied" : "Copy guide"}
                  </Button>
                  <Button variant="secondary" className="rounded-2xl bord
                    <Download className="mr-2 h-4 w-4" />
                    Export .md
                  </Button>
                  <Button variant="outline" className="rounded-2xl border-
                    <Plus className="mr-2 h-4 w-4" />
                    Add step
                  </Button>
                  <Button variant="outline" className="rounded-2xl border-
                    <RefreshCw className="mr-2 h-4 w-4" />
```

```
                Refresh saved
              </Button>
              <Button variant="outline" className="rounded-2xl border-
                <RotateCcw className="mr-2 h-4 w-4" />
                Reset
              </Button>
            </div>
          </CardContent>
        </Card>
      </motion.div>

      <div className="mb-6 grid gap-6 lg:grid-cols-[1.1fr_0.9fr]">
        <Card className={`${glass} rounded-[28px]`}>
          <CardHeader>
            <div className="flex items-center gap-3">
              <div className="flex h-10 w-10 items-center justify-cen
                <Database className="h-5 w-5 text-slate-800" />
              </div>
              <div>
                <CardTitle className="text-slate-900">Backend studio<,
                <CardDescription className="text-slate-700">
                  This app supports local persistence and remote API :
                </CardDescription>
              </div>
            </div>
          </CardHeader>
          <CardContent className="space-y-4">
            <div className="grid gap-4 sm:grid-cols-[0.8fr_1.2fr]">
              <div className="space-y-2">
                <Label className="text-slate-800">Storage mode</Label:
                <Select value={backendMode} onValueChange={(value: Ba
                  <SelectTrigger className="rounded-2xl border-white/
                    <SelectValue />
                  </SelectTrigger>
                  <SelectContent>
                    <SelectItem value="local">Local storage</SelectIt
                    <SelectItem value="remote">Remote API</SelectItem:
                  </SelectContent>
                </Select>
              </div>
              <div className="space-y-2">
                <Label className="text-slate-800">API base URL</Label:
                <Input
                  className="rounded-2xl border-white/30 bg-white/18 
                  placeholder="http://localhost:3001"
                  value={apiBaseUrl}
                  onChange={(e) => setApiBaseUrl(e.target.value)}
                  disabled={backendMode === "local"}
```

```
                    />
                  </div>
                </div>

                <div className="grid gap-3 sm:grid-cols-3">
                  <div className="rounded-[24px] border border-white/30 b
                    <div className="mb-2 flex items-center gap-2 text-sm
                      {backendMode === "local" ? <HardDrive className="h-
                      Active mode
                    </div>
                    <div className="text-sm leading-6 text-slate-700">{ba
                  </div>
                  <div className="rounded-[24px] border border-white/30 b
                    <div className="mb-2 flex items-center gap-2 text-sm
                      {connectionState === "online" ? <Wifi className="h-
                      Connection
                    </div>
                    <div className="text-sm leading-6 text-slate-700 capi
                  </div>
                  <div className="rounded-[24px] border border-white/30 b
                    <div className="mb-2 flex items-center gap-2 text-sm
                      <Server className="h-4 w-4" />
                      Sync status
                    </div>
                    <div className="text-sm leading-6 text-slate-700 capi
                  </div>
                </div>

                <div className="flex flex-wrap gap-2">
                  <Button className="rounded-2xl border border-white/35 b
                    <Wifi className="mr-2 h-4 w-4" />
                    {isTestingConnection ? "Testing..." : "Test connectio
                  </Button>
                  <Button className="rounded-2xl border border-white/35 b
                    <Save className="mr-2 h-4 w-4" />
                    Save current guide
                  </Button>
                  <Button variant="outline" className="rounded-2xl border
                    <RefreshCw className="mr-2 h-4 w-4" />
                    {isLoadingGuides ? "Refreshing..." : "Refresh guides"
                  </Button>
                </div>

                <div className="rounded-[24px] border border-dashed borde
                  <div className="mb-2 flex items-center gap-2 text-sm fo
                    <AlertCircle className="h-4 w-4" />
                    Expected API shape
                  </div>
```

```
            <div className="space-y-1 text-sm leading-6 text-slate-
              <div><code>GET /api/guides</code> → array of saved gu
              <div><code>POST /api/guides</code> → upsert one guide
              <div><code>DELETE /api/guides/:id</code> → delete one
              <div><code>GET /api/health</code> → optional health cl
            </div>
          </div>

          <div className="rounded-[24px] border border-white/30 bg-
            <div className="text-sm font-semibold text-slate-900">S
            <div className="mt-1 text-sm leading-6 text-slate-700">
          </div>
        </CardContent>
      </Card>

      <Card className={`${glass} rounded-[28px]`}>
        <CardHeader>
          <div className="flex items-center gap-3">
            <div className="flex h-10 w-10 items-center justify-cen
              <FolderOpen className="h-5 w-5 text-slate-800" />
            </div>
            <div>
              <CardTitle className="text-slate-900">Saved guides</C
              <CardDescription className="text-slate-700">Open, reu
            </div>
          </div>
        </CardHeader>
        <CardContent>
          <ScrollArea className="h-[420px] pr-4">
            <div className="space-y-3">
              {savedGuides.length === 0 ? (
                <div className="rounded-[24px] border border-dashed
                  <div className="text-sm font-medium text-slate-80
                  <div className="mt-1 text-sm leading-6 text-slate-
                </div>
              ) : (
                savedGuides.map((record) => (
                  <div key={record.id} className="rounded-[24px] bo
                    <div className="flex items-start justify-betwee
                      <div className="min-w-0">
                        <div className="truncate text-sm font-semib
                        <div className="mt-1 text-xs text-slate-600
                        <div className="mt-2 text-xs text-slate-600
                      </div>
                      <div className="flex flex-wrap gap-2">
                        <Button size="sm" className="rounded-xl bor
                          Open
                        </Button>
```

```
                        <Button size="sm" variant="outline" classNa
                          <Trash className="h-4 w-4" />
                        </Button>
                      </div>
                    </div>
                  </div>
                ))
              )}
            </div>
          </ScrollArea>
        </CardContent>
      </Card>
    </div>

    <div className="mb-6 grid gap-6 lg:grid-cols-[1fr_1fr]">
      <Card className={`${glass} rounded-[28px]`}>
        <CardHeader>
          <div className="flex items-center gap-3">
            <div className="flex h-10 w-10 items-center justify-cen
              <BrainCircuit className="h-5 w-5 text-slate-800" />
            </div>
            <div>
              <CardTitle className="text-slate-900">Gemini automati
              <CardDescription className="text-slate-700">
                Gemini can now do more than wording help. It can an
              </CardDescription>
            </div>
          </div>
        </CardHeader>
        <CardContent className="space-y-4">
          <div className="grid gap-4 sm:grid-cols-[1.1fr_0.9fr]">
            <div className="space-y-2">
              <Label className="text-slate-800">Gemini API key</Lab
              <div className="flex gap-2">
                <Input
                  type={showGeminiKey ? "text" : "password"}
                  className="rounded-2xl border-white/30 bg-white/1
                  placeholder="Paste your Gemini API key"
                  value={geminiApiKey}
                  onChange={(e) => setGeminiApiKey(e.target.value)}
                />
                <Button
                  type="button"
                  variant="outline"
                  className="rounded-2xl border-white/35 bg-white/1
                  onClick={() => setShowGeminiKey((prev) => !prev)}
                >
                  {showGeminiKey ? "Hide" : "Show"}
```

```
            </Button>
          </div>
        </div>
        <div className="space-y-2">
          <Label className="text-slate-800">Gemini model</Label:
          <Select value={geminiModel} onValueChange={setGeminiM
            <SelectTrigger className="rounded-2xl border-white/
              <SelectValue />
            </SelectTrigger>
            <SelectContent>
              <SelectItem value="gemini-2.5-flash">gemini-2.5-f
              <SelectItem value="gemini-2.5-pro">gemini-2.5-pro
              <SelectItem value="gemini-2.0-flash">gemini-2.0-f
            </SelectContent>
          </Select>
        </div>
      </div>

      <div className="grid gap-3 sm:grid-cols-3">
        <div className="rounded-[24px] border border-yellow-200,
          <div className="mb-2 flex items-center gap-2 text-sm
            <KeyRound className="h-4 w-4" />
            Key status
          </div>
          <div className="text-sm leading-6 text-slate-700">{is
        </div>
        <div className="rounded-[24px] border border-emerald-20
          <div className="mb-2 flex items-center gap-2 text-sm
            <Bot className="h-4 w-4" />
            AI mode
          </div>
          <div className="text-sm leading-6 text-slate-700">{us
        </div>
        <div className="rounded-[24px] border border-white/30 b
          <div className="mb-2 flex items-center gap-2 text-sm
            {isAiWorking ? <Loader2 className="h-4 w-4 animate-
            AI status
          </div>
          <div className="text-sm leading-6 text-slate-700">{is
        </div>
      </div>

      <div className="flex flex-wrap gap-2">
        <Button
          className="rounded-2xl border border-white/35 bg-whit
          onClick={() => setUseGeminiAutomation((prev) => !prev
        >
          <Bot className="mr-2 h-4 w-4" />
```

```
            {useGeminiAutomation ? "Use local automation" : "Use (
          </Button>
          <Button
            variant="outline"
            className="rounded-2xl border-white/35 bg-white/12 te
            onClick={testGeminiConnection}
            disabled={isAiWorking}
          >
            {isAiWorking ? <Loader2 className="mr-2 h-4 w-4 anima
            Test Gemini
          </Button>
          <Button
            variant="outline"
            className="rounded-2xl border-white/35 bg-white/12 te
            onClick={() => {
              setGeminiApiKey("");
              setUseGeminiAutomation(false);
              setAiStatusMessage("Cleared the Gemini API key and 
            }}
          >
            Clear key
          </Button>
        </div>

        <div className="grid gap-2 sm:grid-cols-2">
          <Button
            className="rounded-2xl border border-white/35 bg-whit
            onClick={autoBuildGuideMeta}
            disabled={isAiWorking}
          >
            {isAiWorking ? <Loader2 className="mr-2 h-4 w-4 anima
            Fill guide details
          </Button>
          <Button
            className="rounded-2xl border border-white/35 bg-whit
            onClick={autoBuildGuideFromImages}
            disabled={isAiWorking}
          >
            {isAiWorking ? <Loader2 className="mr-2 h-4 w-4 anima
            Build guide from images
          </Button>
        </div>

        <div className="grid gap-3 sm:grid-cols-2">
          <div className="rounded-[24px] border border-white/30 b
            <div className="mb-1 flex items-center gap-2 text-sm 
              <ScanText className="h-4 w-4" />
              What Gemini automates now
```

```
                </div>
                <div className="text-sm leading-6 text-slate-700">
                  It can turn rough notes into steps, analyze attache
                </div>
              </div>
              <div className="rounded-[24px] border border-white/30 b
                <div className="mb-1 flex items-center gap-2 text-sm
                  <AlertCircle className="h-4 w-4" />
                  What it still does not automate
                </div>
                <div className="text-sm leading-6 text-slate-700">
                  It does not click through apps, control your comput
                </div>
              </div>
            </div>

            <div className="rounded-[24px] border border-amber-200/60
              <div className="mb-2 flex items-center gap-2 text-sm fo
                <ShieldAlert className="h-4 w-4" />
                Security note
              </div>
              <div className="text-sm leading-6 text-amber-900/90">
                This version stores your Gemini API key in local brow
              </div>
            </div>

            <div className="rounded-[24px] border border-white/30 bg-
              <div className="text-sm font-semibold text-slate-900">G
              <div className="mt-1 text-sm leading-6 text-slate-700">
            </div>
          </CardContent>
        </Card>

        <Card className={`${glass} rounded-[28px]`}>
          <CardHeader>
            <div className="flex items-center gap-3">
              <div className="flex h-10 w-10 items-center justify-cen
                <Bot className="h-5 w-5 text-slate-800" />
              </div>
              <div>
                <CardTitle className="text-slate-900">Automation stud
                <CardDescription className="text-slate-700">
                  Paste rough notes, attach screenshots, and let the
                </CardDescription>
              </div>
            </div>
          </CardHeader>
          <CardContent className="space-y-4">
```

```
                <input
                  ref={bulkImageInputRef}
                  type="file"
                  accept="image/*"
                  multiple
                  className="hidden"
                  onChange={(e) => {
                    void handleBulkScreenshotsUpload(e.target.files);
                    e.currentTarget.value = "";
                  }}
                />

                <div className="rounded-[24px] border border-dashed borde
                  <div className="flex flex-col gap-4 sm:flex-row sm:item
                    <div>
                      <div className="text-sm font-semibold text-slate-90
                      <div className="mt-1 text-sm leading-6 text-slate-7
                        Upload multiple screenshots at once, drag them in
                      </div>
                    </div>
                    <div className="flex flex-wrap gap-2">
                      <Button
                        className="rounded-2xl border border-white/35 bg-
                        onClick={() => bulkImageInputRef.current?.click()
                        disabled={isAiWorking}
                      >
                        {isAiWorking ? <Loader2 className="mr-2 h-4 w-4 a
                        Upload screenshots
                      </Button>
                      <Button
                        variant="outline"
                        className="rounded-2xl border-white/35 bg-white/1
                        onClick={autoBuildGuideFromImages}
                        disabled={isAiWorking || imageSteps.length === 0}
                      >
                        {isAiWorking ? <Loader2 className="mr-2 h-4 w-4 a
                        Build ordered guide
                      </Button>
                    </div>
                  </div>
                </div>

                {imageSteps.length > 0 && (
                  <div className="rounded-[24px] border border-white/30 b
                    <div className="mb-3 flex items-center justify-betwee
                      <div>
                        <div className="text-sm font-semibold text-slate-
                        <div className="mt-1 text-sm leading-6 text-slate-
```

```
                    </div>
                    <Badge className="rounded-full border border-white/
                      {imageSteps.length} image{imageSteps.length === 1
                    </Badge>
                  </div>

                  <div className="grid gap-3 sm:grid-cols-2 xl:grid-col
                    {imageSteps.map((step, index) => (
                      <div
                        key={step.id}
                        draggable
                        onDragStart={() => handleSequenceDragStart(step
                        onDragOver={(e) => e.preventDefault()}
                        onDrop={() => handleSequenceDrop(step.id)}
                        onDragEnd={() => setDraggedStepId(null)}
                        className={`rounded-[22px] border p-3 backdrop-
                      >
                        <div className="mb-2 flex items-center justify-
                          <div className="flex items-center gap-2 text-
                            <GripVertical className="h-4 w-4 text-slate
                            Step {index + 1}
                          </div>
                          <span className="text-xs text-slate-500">Drag
                        </div>
                        <div className="overflow-hidden rounded-[16px]
                          <img src={step.image?.src} alt={step.image?.n
                        </div>
                        <div className="mt-2 truncate text-sm font-medi
                        <div className="mt-1 text-xs text-slate-600">{s
                      </div>
                    ))}
                  </div>
                </div>
              )}

              <div className="space-y-2">
                <Label className="text-slate-800">Paste a rough workflo
                <Textarea
                  className="min-h-[120px] rounded-2xl border-white/30
                  placeholder={"Example:\nOpen OMIS\nReview referral\nC
                  value={outlineInput}
                  onChange={(e) => setOutlineInput(e.target.value)}
                />
              </div>
              <div className="grid gap-4 sm:grid-cols-[0.9fr_1.1fr]">
                <div className="space-y-2">
                  <Label className="text-slate-800">Apply generated ste
                  <Select value={automationMode} onValueChange={(value:
```

```
                <SelectTrigger className="rounded-2xl border-white/
                  <SelectValue />
                </SelectTrigger>
                <SelectContent>
                  <SelectItem value="append">Append to current step
                  <SelectItem value="replace">Replace current steps
                </SelectContent>
              </Select>
            </div>
            <div className="grid gap-2 sm:grid-cols-2">
              <Button className="rounded-2xl border border-white/35
                {isAiWorking ? <Loader2 className="mr-2 h-4 w-4 ani
                Generate steps
              </Button>
              <Button variant="outline" className="rounded-2xl bord
                {isAiWorking ? <Loader2 className="mr-2 h-4 w-4 ani
                Polish all steps
              </Button>
            </div>
          </div>

          <div className="grid gap-3 sm:grid-cols-3">
            <div className="rounded-[24px] border border-white/30 b
              <div className="mb-1 flex items-center gap-2 text-sm
                <Images className="h-4 w-4" />
                Screenshot-driven
              </div>
              <div className="text-sm leading-6 text-slate-700">Att
            </div>
            <div className="rounded-[24px] border border-white/30 b
              <div className="mb-1 flex items-center gap-2 text-sm
                <Clock3 className="h-4 w-4" />
                Metadata automation
              </div>
              <div className="text-sm leading-6 text-slate-700">Gen
            </div>
            <div className="rounded-[24px] border border-white/30 b
              <div className="mb-1 flex items-center gap-2 text-sm
                <Package className="h-4 w-4" />
                Whole-guide creation
              </div>
              <div className="text-sm leading-6 text-slate-700">Bui
            </div>
          </div>
        </CardContent>
      </Card>
    </div>
```

```
<Tabs value={activeTab} onValueChange={setActiveTab} className="
  <TabsList className="grid w-full max-w-md grid-cols-2 rounded-
    <TabsTrigger value="builder" className="rounded-[18px] data-
      Builder
    </TabsTrigger>
    <TabsTrigger value="preview" className="rounded-[18px] data-
      Preview
    </TabsTrigger>
  </TabsList>

  <TabsContent value="builder" className="mt-0">
    <div className="grid gap-6 lg:grid-cols-[1fr_1fr]">
      <Card className={`${glass} rounded-[28px]`}>
        <CardHeader>
          <CardTitle className="text-slate-900">Guide details</(
          <CardDescription className="text-slate-700">Start wit
        </CardHeader>
        <CardContent className="space-y-4">
          <div className="space-y-2">
            <Label className="text-slate-800">Guide title</Labe
            <Input
              className="rounded-2xl border-white/30 bg-white/1
              placeholder="Example: How to Process a New Case"
              value={guide.title}
              onChange={(e) => updateGuide("title", e.target.va
            />
          </div>

          <div className="grid gap-4 sm:grid-cols-2">
            <div className="space-y-2">
              <Label className="text-slate-800">Audience</Label
              <Input
                className="rounded-2xl border-white/30 bg-white,
                placeholder="Example: New hires"
                value={guide.audience}
                onChange={(e) => updateGuide("audience", e.targ
              />
            </div>
            <div className="space-y-2">
              <Label className="text-slate-800">Estimated time<,
              <Input
                className="rounded-2xl border-white/30 bg-white,
                placeholder="Example: 10 minutes"
                value={guide.estimatedTime}
                onChange={(e) => updateGuide("estimatedTime", e
              />
            </div>
          </div>
```

```
            <div className="grid gap-2 sm:grid-cols-2">
              <Button className="rounded-2xl border border-white/
                {isAiWorking ? <Loader2 className="mr-2 h-4 w-4 a
                Fill details with AI
              </Button>
              <Button variant="outline" className="rounded-2xl bo
                {isAiWorking ? <Loader2 className="mr-2 h-4 w-4 a
                Build from screenshots
              </Button>
            </div>

            <div className="space-y-2">
              <Label className="text-slate-800">Purpose</Label>
              <Textarea
                className="min-h-[100px] rounded-2xl border-white,
                placeholder="What is this guide for?"
                value={guide.purpose}
                onChange={(e) => updateGuide("purpose", e.target.
              />
            </div>

            <div className="space-y-2">
              <Label className="text-slate-800">Materials needed<,
              <Input
                className="rounded-2xl border-white/30 bg-white/1
                placeholder="Example: Access to system, forms, ca
                value={guide.materials}
                onChange={(e) => updateGuide("materials", e.targe
              />
            </div>

            <div className="space-y-2">
              <Label className="text-slate-800">Extra notes</Labe
              <Textarea
                className="min-h-[90px] rounded-2xl border-white/
                placeholder="Optional notes, reminders, escalatio
                value={guide.notes}
                onChange={(e) => updateGuide("notes", e.target.va
              />
            </div>
          </CardContent>
        </Card>

        <Card className={`${glass} rounded-[28px]`}>
          <CardHeader>
            <div className="flex items-center justify-between gap
              <div>
```

```
                <CardTitle className="text-slate-900">Step builde
                <CardDescription className="text-slate-700">Add, 
              </div>
              <Button className="rounded-2xl border border-white/
                <Plus className="mr-2 h-4 w-4" />
                Add
              </Button>
            </div>
          </CardHeader>
          <CardContent>
            <ScrollArea className="h-[1000px] pr-4">
              <div className="space-y-4">
                {guide.steps.map((step, index) => (
                  <motion.div
                    key={step.id}
                    layout
                    initial={{ opacity: 0, y: 6 }}
                    animate={{ opacity: 1, y: 0 }}
                    className={`${glassSoft} rounded-[24px] p-4`}
                  >
                    <div className="mb-3 flex items-center justif
                      <div className="flex items-center gap-2">
                        <div className="rounded-2xl border border
                          <GripVertical className="h-4 w-4" />
                        </div>
                        <Badge className="rounded-full border bor
                          Step {index + 1}
                        </Badge>
                        {step.image?.src && (
                          <Badge className="rounded-full border b
                            Screenshot ready
                          </Badge>
                        )}
                      </div>
                      <div className="flex flex-wrap items-center
                        <Button
                          variant="outline"
                          size="sm"
                          className="rounded-xl border-white/30 b
                          onClick={() => autoFillSingleStep(step,
                          disabled={isAiWorking}
                        >
                          {isAiWorking ? <Loader2 className="mr-1
                          Auto-fill text
                        </Button>
                        <Button
                          variant="outline"
                          size="sm"
```

```
                    className="rounded-xl border-white/30 b
                    onClick={() => autoAnalyzeStepImage(ste
                    disabled={isAiWorking || !step.image?.s
                  >
                    {isAiWorking ? <Loader2 className="mr-1
                    Analyze image
                  </Button>
                  <Button variant="outline" size="sm" class
                  <Button variant="outline" size="sm" class
                  <Button variant="outline" size="sm" class
                    <Trash2 className="h-4 w-4" />
                  </Button>
                </div>
              </div>

              <div className="space-y-3">
                <div className="space-y-2">
                  <Label className="text-slate-800">Step ti
                  <Input
                    className="rounded-2xl border-white/30 
                    placeholder="Example: Review the referr
                    value={step.title}
                    onChange={(e) => updateStep(step.id, "t
                  />
                </div>
                <div className="space-y-2">
                  <Label className="text-slate-800">Main in
                  <Textarea
                    className="min-h-[90px] rounded-2xl bor
                    placeholder="Explain what to do in this
                    value={step.instruction}
                    onChange={(e) => updateStep(step.id, "i
                  />
                </div>
                <div className="grid gap-3 sm:grid-cols-2">
                  <div className="space-y-2">
                    <Label className="text-slate-800">Helpf
                    <Textarea
                      className="min-h-[80px] rounded-2xl b
                      placeholder="Optional shortcut or rem
                      value={step.tip}
                      onChange={(e) => updateStep(step.id, 
                    />
                  </div>
                  <div className="space-y-2">
                    <Label className="text-slate-800">Watch
                    <Textarea
                      className="min-h-[80px] rounded-2xl b
```

```
                placeholder="Common mistake or cautio
                value={step.caution}
                onChange={(e) => updateStep(step.id, '
              />
            </div>
          </div>

          <div className="rounded-[24px] border borde
            <div className="mb-3 flex flex-wrap items-
              <div>
                <div className="text-sm font-semibold
                <div className="text-sm text-slate-70
              </div>
              <Label className="inline-flex cursor-po
                <Upload className="mr-2 h-4 w-4" />
                Upload image
                <input
                  type="file"
                  accept="image/*"
                  className="hidden"
                  onChange={(e) => onStepImageUpload(
                />
              </Label>
            </div>

            {step.image?.src ? (
              <div className="space-y-3">
                <div className="relative overflow-hid
                  <img src={step.image.src} alt={step
                  <button
                    type="button"
                    className="absolute right-3 top-3
                    onClick={() => updateStep(step.id
                  >
                    <X className="h-4 w-4" />
                  </button>
                </div>
                <div className="grid gap-3 sm:grid-co
                  <div className="rounded-[20px] bord
                    <div className="text-xs uppercase
                    <div className="mt-1 break-all fo
                  </div>
                  <div className="space-y-2">
                    <Label className="text-slate-800"
                    <Input
                      className="rounded-2xl border-w
                      placeholder="Example: Screensho
                      value={step.image.caption}
```

```
                                        onChange={(e) =>
                                          updateStep(step.id, "image", 
                                            ...step.image!,
                                            caption: e.target.value,
                                          })
                                        }
                                      />
                                    </div>
                                  </div>
                                </div>
                              ) : (
                                <div className="flex min-h-[150px] flex-
                                  <ImagePlus className="mb-3 h-8 w-8 te
                                  <div className="text-sm font-medium t
                                  <div className="mt-1 max-w-sm text-sm
                                    Add screenshots, photos, or visual
                                  </div>
                                </div>
                              )}
                            </div>
                          </div>
                        </motion.div>
                      ))}
                    </div>
                  </ScrollArea>
                </CardContent>
              </Card>
            </div>
          </TabsContent>

          <TabsContent value="preview" className="mt-0">
            <div className="grid gap-6 lg:grid-cols-[1.15fr_0.85fr]">
              <Card className={`${glass} rounded-[28px]`}>
                <CardHeader>
                  <div className="flex flex-wrap items-center justify-b
                    <div>
                      <CardTitle className="text-slate-900">Guide previ
                      <CardDescription className="text-slate-700">Readal
                    </div>
                    <div className="flex gap-2">
                      <Button className="rounded-2xl border border-whit
                        <Copy className="mr-2 h-4 w-4" />
                        {copied ? "Copied" : "Copy"}
                      </Button>
                      <Button variant="secondary" className="rounded-2x
                        <Download className="mr-2 h-4 w-4" />
                        Export
                      </Button>
```

```
                    </div>
                  </div>
                </CardHeader>
                <CardContent>
                  <div className="rounded-[28px] border border-white/35
                    <div className="space-y-5">
                      <div>
                        <h1 className="text-3xl font-semibold tracking-
                        <p className="mt-2 text-sm text-slate-600">Fast
                      </div>

                      {(guide.audience || guide.purpose || guide.estima
                        <div className="grid gap-3 sm:grid-cols-2">
                          {guide.audience && (
                            <div className="rounded-[24px] border borde
                              <div className="text-xs font-medium upper
                              <div className="mt-1 font-medium text-sla
                            </div>
                          )}
                          {guide.estimatedTime && (
                            <div className="rounded-[24px] border borde
                              <div className="text-xs font-medium upper
                              <div className="mt-1 font-medium text-sla
                            </div>
                          )}
                          {guide.materials && (
                            <div className="rounded-[24px] border borde
                              <div className="text-xs font-medium upper
                              <div className="mt-1 font-medium text-sla
                            </div>
                          )}
                          {guide.purpose && (
                            <div className="rounded-[24px] border borde
                              <div className="text-xs font-medium upper
                              <div className="mt-1 leading-7 text-slate
                            </div>
                          )}
                          {guide.notes && (
                            <div className="rounded-[24px] border borde
                              <div className="text-xs font-medium upper
                              <div className="mt-1 leading-7 text-slate
                            </div>
                          )}
                        </div>
                      )}

                      <Separator className="bg-white/35" />
```

```
<div className="space-y-4">
  <div className="flex items-center gap-2 text-lg
    <CheckCircle2 className="h-5 w-5" />
    Steps
  </div>
  {guide.steps.map((step, index) => (
    <div key={step.id} className="rounded-[24px] |
      <div className="mb-3 flex items-center gap-
        <div className="flex h-10 w-10 items-cent
          {index + 1}
        </div>
        <h3 className="text-lg font-semibold text
      </div>
      {step.instruction && <p className="leading-

      {step.image?.src && (
        <div className="mt-4 overflow-hidden roun
          <img src={step.image.src} alt={step.ima
          {(step.image.caption || step.image.name
            <div className="border-t border-white,
              <span className="font-medium text-s
            </div>
          )}
        </div>
      )}

      {(step.tip || step.caution) && (
        <div className="mt-4 grid gap-3 sm:grid-c
          {step.tip && (
            <div className="rounded-[22px] border
              <div className="mb-1 text-xs font-m
              <div className="text-sm leading-6 t
            </div>
          )}
          {step.caution && (
            <div className="rounded-[22px] border
              <div className="mb-1 text-xs font-m
              <div className="text-sm leading-6 t
            </div>
          )}
        </div>
      )}
    </div>
```

Done — I added the **drag-and-drop screenshot ordering** layer.

What changed:

- after multi-upload, screenshots now stay in a **sequence panel**
- each screenshot card is **draggable**
- you can reorder them before guide generation
- there is now a dedicated **Build ordered guide** button
- upload no longer immediately forces the build step
- the status text now tells you to reorder first, then build

So the flow is now:

1. upload multiple screenshots
2. drag them into the right order
3. click **Build ordered guide**
4. Gemini generates the guide from that sequence

That should make the screenshot-to-guide workflow feel much more controlled.